Tutorial: Introduction to Graph and Terminology

For Data Structure Students

Learning goals: Understand what a graph is, why graphs are used, and the basic terms needed to solve graph problems in Data Structures.

1. What is a Graph?

A graph is a non-linear data structure used to represent relationships between objects. In a graph, the objects are called vertices (or nodes) and the relationships between them are called edges.

We usually write a graph as G = (V, E), where V is the set of vertices and E is the set of edges.

- In a social network, each person can be a vertex and friendship can be an edge.
- In a map, each city can be a vertex and a road between cities can be an edge.
- In a computer network, each device can be a vertex and a connection can be an edge.

2. Why Do We Study Graphs?

- Graphs model real-world problems naturally.
- Many important algorithms use graphs, such as shortest path, traversal, spanning tree, and network flow.
- Graphs appear in routing, recommendation systems, scheduling, compilers, circuits, and operating systems.

3. Basic Parts of a Graph

Term	Meaning	Quick Example
Vertex / Node	An individual object or point in a graph.	A, B, C, D
Edge	A connection between two vertices.	(A, B)
Directed Edge	An edge with direction from one vertex to another.	A → B
Undirected Edge	An edge with no direction.	A — B
Weight	A value attached to an edge such as cost, distance, or time.	Road length = 10 km

4. Types of Graphs

4.1 Undirected Graph

In an undirected graph, edges do not have direction. If A is connected to B, then B is also connected to A.

4.2 Directed Graph (Digraph)

In a directed graph, each edge has a direction. The edge A → B means movement or relation from A to B only.

4.3 Weighted Graph

A weighted graph stores a weight on each edge. Weights may represent distance, cost, time, or capacity.

4.4 Unweighted Graph

In an unweighted graph, all edges are treated equally, so no numerical value is stored on them.

4.5 Simple Graph, Multigraph, and Pseudograph

- Simple graph: no self-loops and no multiple edges between the same pair of vertices.
- Multigraph: may contain more than one edge between the same pair of vertices.
- Pseudograph: may contain self-loops.

5. Important Graph Terminology

Term	Meaning	Quick Example
Adjacent Vertices	Two vertices joined directly by an edge.	A and B are adjacent if edge AB exists.
Incident Edge	An edge connected to a vertex.	Edge AB is incident on A and B.
Degree	Number of edges connected to a vertex in an undirected graph.	deg(A) = 3
In-degree	Number of incoming edges to a vertex in a directed graph.	Two arrows enter C
Out-degree	Number of outgoing edges from a vertex in a directed graph.	Three arrows leave C
Self-loop	An edge from a vertex to itself.	A → A
Parallel Edges	Multiple edges joining the same pair of vertices.	Two different edges between A and B

Degree Rule

For an undirected graph, the sum of the degrees of all vertices is equal to twice the number of edges. This is called the Handshaking Lemma.

Sum of degrees of all vertices = 2 × number of edges

6. Path, Cycle, and Connectivity

Term	Meaning	Quick Example
Walk	A sequence of vertices and edges where repetition is allowed.	A-B-C-B
Path	A sequence of distinct vertices connected by edges.	A-B-C-D
Simple Path	A path in which no vertex repeats.	P to Q to R
Cycle	A path that starts and ends at the same vertex.	A-B-C-A
Connected Graph	Every vertex can be reached from every other vertex.	One connected component
Disconnected Graph	At least one vertex cannot be reached from another.	Two separate groups of vertices

In directed graphs, the idea of connectivity may be discussed as strongly connected or weakly connected, depending on whether direction is respected during reachability.

7. Special Graphs Commonly Used in Data Structures

- Complete graph: every pair of distinct vertices is connected by an edge.
- Null graph: a graph with vertices but no edges.
- Regular graph: every vertex has the same degree.
- Bipartite graph: vertices can be divided into two sets and every edge joins one set to the other.
- Tree: a connected graph with no cycles.
- Subgraph: a graph formed using a subset of vertices and edges from another graph.

8. Example Graph

Consider the undirected graph with vertices V = {A, B, C, D, E} and edges E = {AB, AC, BC, CD, DE}.

- A is adjacent to B and C.
- Degree of A = 2.
- Degree of C = 3 because C is connected to A, B, and D.
- A-B-C forms a cycle only if the edges AB, BC, and AC all exist. Here they do, so A-B-C-A is a cycle.
- The graph is connected because every vertex can be reached from any other vertex.

9. Graph Representation (Introductory View)

Once we understand graph terminology, the next step is to store graphs in memory. The two most common representations are adjacency matrix and adjacency list.

Representation	Idea	Best Use
Adjacency Matrix	2D array where matrix[i][j] shows whether an edge exists.	Dense graphs and fast edge lookup
Adjacency List	Each vertex stores a list of its neighboring vertices.	Sparse graphs and memory-efficient storage

10. Quick Comparison: Tree vs Graph

Feature	Tree	General Graph
Cycles	Not allowed	May exist
Connectivity	Always connected	May be connected or disconnected
Root	Often defined	Usually not required
Edges	For n vertices, edges = n - 1	No fixed rule

11. Summary

- A graph represents relationships between objects.
- Vertices are the objects; edges are the links between them.
- Graphs may be directed, undirected, weighted, or unweighted.
- Important terms include degree, adjacency, path, cycle, and connected graph.
- Understanding terminology is the foundation for graph traversal and graph algorithms.

12. Practice Questions

1. Define graph, vertex, and edge in your own words.

2. Differentiate between directed and undirected graphs with one example each.

3. What is the degree of a vertex? How is it different from in-degree and out-degree?

4. What is a path? What is a cycle?

5. When is a graph called connected?

6. What is the difference between a tree and a graph?